



400-650-7088

TongLINK/Q-CN V10.0

快速使用手册

版本：10041A01

2025 年 9 月

声明

版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech** 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

本手册版本（10041A01）适用于TongLINK/Q-CN V10.0.4.1。

文档版本	适用产品版本	更新内容	更新日期
10041A01	V10.0.4.1	更新如下内容： 更新版权声明。	2025/9/23
10040A01	V10.0.4.0	新增如下内容： 2.2.1 获取镜像、helm、operator包：新增部署包名称及说明； 2.2.2 上传安装包：新增解压operator包的步骤； 2.2.3 导入镜像：新增镜像导入命令； 2.3.1 组件开启：新增章节； 2.4 方法二：Operator部署：新增章节； 3.1.2 获取安装包：新增安装包及说明； 3.1.3 上传并解压安装包：新增解压命令； 3.1.5 配置JDK：新增章节； 3.1.6 确定集群名称：新增章节； 3.1.7 （可选）安装License Server：新增章节； 3.2 单机部署：新增章节； 3.3.6 服务地址：新增章节。 更新如下内容： 2.3.2.1 最小化部署：修改部署步骤； - 第3章 虚拟机部署：修改描述内容； 3.1.1 运行环境：删除“高性能集群部署”方式介绍； 3.1.4 目录说明：更新目录名称及说明； 3.3.1.1 修改Zookeeper配置文件：更新步骤2中参数及说明； 3.3.1.2 启停Zookeeper：更新启停命令及介绍； 3.3.2 初始化元数据：更新命令及介绍； 3.3.3.2 启停BookKeeper：更新启停命令及介绍； 3.3.4.2 启停Broker：更新启停命令及介绍。	2025/5/23

文档版本	适用产品版本	更新内容	更新日期
10030A01	V10.0.3.0	新增如下内容： 2.1 部署前准备：新增安装License Server 4.5.1.6及以上版本的步骤。 更新如下内容： • 1.1 产品介绍：修改描述； • 2.2.1 获取镜像及helm包：删除License Server镜像包； • 2.2.3 导入镜像：新增私有镜像仓库导入方式说明； • 2.3.2 （可选）配置证书认证：删除内外置License Server配置描述；新增POC模式无需配置认证的说明； • 第3章 虚拟机部署：删除全部章节下重命名TLQ-CN安装目录的步骤；更新安装License Server的版本为4.5.1.6及以上版本。	2024/11/30
10020A01	V10.0.2.0	新增如下内容： • 2.3.1 部署场景说明：新增章节； • 2.3.4 helm部署集群：新增使用内置License Server的说明； • 2.5.3 安装License Server并获取License信息：新增章节； • 2.6.6.2 启停broker：新增启动时出现错误的解决方法。 更新如下内容： • 2.2.1 获取镜像及helm包：更新安装包名称； • 2.2.2 上传安装包：更新安装包名称； • 2.3.3 配置证书认证：更新配置步骤； • 2.5.1 运行环境：更新机器配置建议。	2024/5/31
10010A01	V10.0.1.0	更新如下内容： • 2.3.1 配置证书认证：更新参数名称； • 3.2.5.1 修改配置文件：更新参数名称。	2023/12/29

文档版本	适用产品版本	更新内容	更新日期
10001A01	V10.0.0.1	新增如下内容： • 3.1.1 运行环境：新增三种部署方式介绍； • 3.2 集群部署：新增说明； • 3.2.3.1 修改配置文件：新增server.id的格式说明。	2023/10/16
10000A01	V10.0.0.0	TongLINK/Q-CN V10.0第一次正式发布。	2023/7/14

前言

本文档是TongLINK/Q-CN V10.0产品的用户使用手册之一，提供产品介绍，在K8s和虚拟机上安装部署产品的基本步骤及验证方式。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

• 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

• 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

• 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

1. 掌握Linux系统基本操作；
2. 掌握K8s的搭建和使用；
3. 熟悉Java开发语言。

• 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

• 实践经验

为了最大程度地受益于本文档，建议您具备一定实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

用户使用手册集

TongLINK/Q-CN V10.0为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongLINK/Q-CN_V10.0用户手册导读	提供手册集的目录及手册版本说明。
2	001_TongLINK/Q-CN_V10.0快速使用手册	提供产品介绍，在K8s和虚拟机上安装部署产品的基本步骤及验证方式。

编号	手册集文档	说明
3	002_TongLINK/Q-CN_V10.0配置参数手册	提供配置文件及参数的详细说明。
4	003_TongLINK/Q-CN_V10.0监控管理手册	提供在K8s和虚拟机上安装部署监控管理组件的基本步骤及验证方式。
5	004_TongLINK/Q-CN_V10.0管理控制台使用手册	详细介绍管理控制台的集群管理、集群监控、告警管理和用户管理相关功能的使用说明。
6	005_TongLINK/Q-CN_V10.0命令行使用手册	详细提供产品中主要命令行的作用、相关参数及使用示例。
7	006_TongLINK/Q-CN_V10.0Java客户端开发手册	介绍使用Java客户端生产消费消息的相关概念、配置、参数及示例。
8	007_TongLINK/Q-CN_V10.0Kafka协议部署手册	提供在K8s和虚拟机上安装部署Kafka协议处理程序的基本步骤及验证方式。
9	008_TongLINK/Q-CN_V10.0AMQP协议部署手册	提供在K8s和虚拟机上安装部署AMQP协议处理程序的基本步骤及验证方式。
10	009_TongLINK/Q-CN_V10.0JMS客户端开发手册	介绍如何将适配ActiveMQ的JMS客户端迁移为适配TongLINK/Q-CN的JMS客户端。
11	010_TongLINK/Q-CN_V10.0MQTT协议部署手册	提供在K8s和虚拟机上安装部署MQTT协议处理程序的基本步骤及验证方式。
12	011_TongLINK/Q-CN_V10.0服务端进阶配置指南	提供在K8s和虚拟机上开启TongLINK/Q-CN的国密通信、消息轨迹、消息过滤等高级功能特性的方法。
13	012_TongLINK/Q-CN_V10.0灾备及故障转移手册	介绍使用灾备及故障转移的相关概念、配置、参数及示例。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：www.tongtech.com
- 电话：400-650-7088
- 邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

- 您的姓名

- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

目录

声明	1
版本变更说明	2
前言	5
1. 概述	10
1.1. 产品介绍	10
1.2. 整体架构	10
1.3. 特别说明	11
2. K8s部署	12
2.1. 部署前准备	12
2.2. 安装步骤	12
2.2.1. 获取镜像及helm包	12
2.2.2. 上传安装包	13
2.2.3. 导入镜像	13
2.3. 方法一：Helm部署	13
2.3.1. 组件开启	13
2.3.2. 部署场景说明	14
2.3.2.1. 最小化部署	14
2.3.2.2. 一般部署	15
2.3.2.3. 高性能部署	18
2.3.3. （可选）配置证书认证	19
2.3.4. 创建命名空间	19
2.3.5. helm部署集群	20
2.3.6. 验证集群部署	20
2.4. 方法二：Operator部署	21
2.4.1. 部署operator manager和crds	21
2.4.2. operator部署集群	21
2.4.3. 验证集群部署	22
2.5. 业务验证	22
3. 虚拟机部署	24
3.1. 部署前准备	24
3.1.1. 运行环境	24
3.1.2. 获取安装包	24
3.1.3. 上传并解压安装包	25
3.1.4. 目录说明	25
3.1.5. 配置JDK	26
3.1.6. 确定集群名称	26

3.1.7. (可选) 安装License Server.....	27
3.2. 单机部署	27
3.2.1. (可选) 修改Standalone配置文件.....	27
3.2.2. 启停Standalone服务.....	28
3.3. 集群部署	29
3.3.1. 部署Zookeeper集群	29
3.3.1.1. 修改Zookeeper配置文件.....	29
3.3.1.2. 启停Zookeeper.....	30
3.3.2. 初始化元数据	32
3.3.3. 部署BookKeeper集群.....	33
3.3.3.1. 修改BookKeeper配置文件	33
3.3.3.2. 启停BookKeeper	35
3.3.4. 部署Broker集群.....	37
3.3.4.1. 修改Broker配置文件	37
3.3.4.2. 启停Broker	38
3.3.5. 集群可用性测试.....	40
3.3.6. 服务地址	41
附录 A: 术语说明.....	43

1. 概述

1.1. 产品介绍

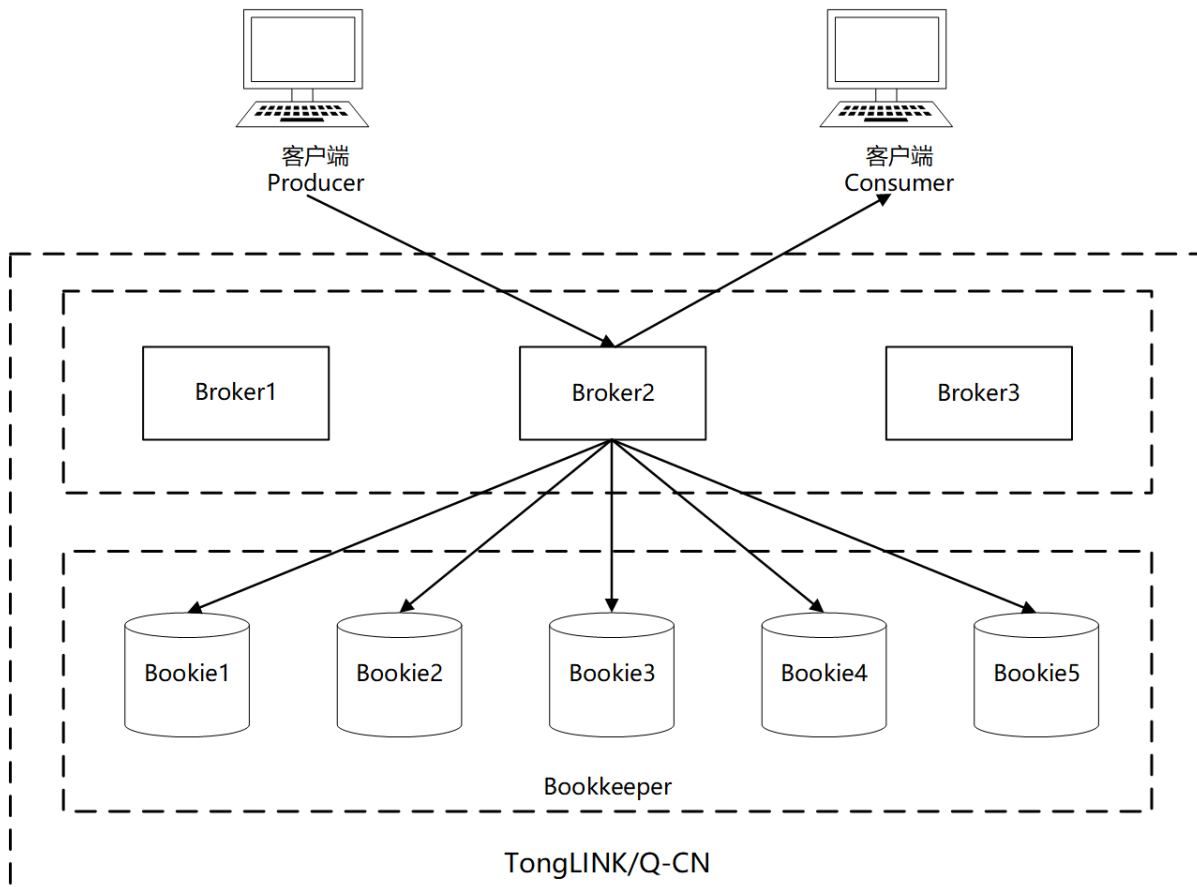
TongLINK/Q-CN是一款云原生分布式消息中间件，助力为实时事件和历史事件构建实时应用。TongLINK/Q-CN计算与存储分离的架构设计，使得它具备极好的云原生和Serverless特性。它拥有原生Java、C++、Python、Go等多种API，同时支持HTTP协议，可为分布式应用系统提供异步解耦和削峰填谷的能力，具备互联网应用所需的海量消息堆积、高吞吐、可靠重试等特性。

TongLINK/Q-CN具有如下优点：

- 采用存储和计算分离的架构，支持实时水平动态无限扩容；
- 无状态的Broker支持快速扩展或收缩集群，实现动态分配；
- 使用分片存储架构，支持数据的独立扩展和快速恢复；
- 节点对等保证数据传输的高容错。

1.2. 整体架构

TongLINK/Q-CN使用计算与存储分离的云原生架构：上层是无状态Broker，负责消息分发和服务；下层是有状态持久化存储层，由Bookie节点组成。产品整体架构图如下所示：



1.3. 特别说明

TongLINK/Q-CN支持在K8s和虚拟机两种不同的环境下部署，以下分别介绍在两种不同环境下的部署和验证方式。此手册将介绍怎样快速搭建一个基础集群，用户可以使用基础集群体验集群的基本功能。

2. K8s部署

2.1. 部署前准备

在K8s上部署TongLINK/Q-CN集群前，请先做如下准备：

1. 安装License Server 4.5.1.6及以上版本，详细安装步骤请参考《License Server 安装部署指南》；
2. 根据用户环境，选择安装对应K8s版本（标准开源k8s版本，如果购买的是企业版，在安装过程中略有差异），版本为V1.21及以上；

当前支持的K8s平台有：标准K8s，Openshift管理的K8s，Rancher管理的K8s，道客云平台的K8s；

3. 根据用户使用的存储类型，创建好对应的StorageClass，对于读写性能有较高要求的场景，需单独创建一个SSD对应的StorageClass；
4. 安装Docker；
5. 安装kubectl；
6. 安装helm。

2.2. 安装步骤

2.2.1. 获取镜像及helm包

TongLINK/Q-CN产品为不同的架构平台提供不同的镜像包，安装之前请查看机器属于哪种架构。如果机器是arm架构服务器，用TongLINK/Q-CN的arm架构镜像包；x86_64架构服务器则用TongLINK/Q-CN的x86_64架构镜像包。TongLINK/Q-CN的镜像、helm及operator包如下：

安装包名称	说明
tonglinkq-cn-10.0.x.tgz	helm部署包，用于部署TongLINK/Q-CN的chart包。
tlq-cn-base_*_10.0.x.x.tar	tonglinkq镜像包，TongLINK/Q-CN核心镜像包，用于启动zk、broker、bookie组件的镜像包。其中*为不同架构平台。不使用kafka、amqp、mqtt协议时，请下载此包。
tlq-cn-base-protocols_*_10.0.x.x.tar	tonglinkq镜像包，TongLINK/Q-CN核心镜像包，用于启动zk、broker、bookie组件的镜像包。其中*为不同架构平台。包含了kafka、amqp、mqtt协议的镜像包。使用这些协议时，请下载此包。
tlq-cn-operator_*_10.0.x.x.tar	tonglinkq服务端operator manager核心镜像。其中*为不同架构平台。
tlq-cn-kube-rbac-proxy_*_10.0.x.x.tar	tonglinkq服务端operator kube rbac proxy镜像。其中*为不同架构平台。

安装包名称	说明
tonglinkq-cn-operator-10.0.x.tgz	operator部署包，用于部署TongLINK/Q-CN的operator的yaml文件压缩包。

2.2.2. 上传安装包

1. 获取产品镜像及helm包，将获取到的安装包上传至已部署K8s的私有云服务器自定义目录下。
2. 解压operator部署包。

在步骤1中上传安装包的路径下执行如下命令解压operator部署所需的压缩包：

```
tar zxvf tonglinkq-cn-operator-10.0.x.tgz
```

3. 解压helm部署包。

在步骤1中上传安装包的路径下执行如下命令解压helm部署所需的chart包：

```
tar zxvf tonglinkq-cn-10.0.x.tgz
```

2.2.3. 导入镜像

用户可以使用私有镜像仓库方式管理镜像包，把镜像上传至私有镜像仓库。

若用户没有私有镜像仓库，那么则需要把镜像导入到k8s的每一个node节点。下面介绍该方式导入镜像的具体操作：

使用如下命令导入“[上传安装包](#)”中上传的镜像文件。

- 不使用kafka、ampq、mqtt协议，导入tlq-cn-base_*_10.0.x.x.tar镜像；
- 使用kafka、ampq、mqtt协议，导入tlq-cn-base-protocols_*_10.0.x.x.tar镜像。

```
docker load -i tlq-cn-base_*_10.0.x.x.tar （不使用协议）
```

```
docker load -i tlq-cn-base-protocols_*_10.0.x.x.tar （使用协议）
```

```
docker load -i tlq-cn-kube-rbac-proxy_*_10.0.x.x.tar
```

```
docker load -i tlq-cn-operator_*_10.0.x.x.tar
```

注意：k8s的每一个准备被调度的node节点都需要导入。

本手册提供[Helm部署](#)和[Operator部署](#)两种在K8s上部署TongLINK/Q-CN集群的方法，详细介绍请参考以下章节。

2.3. 方法一：Helm部署

2.3.1. 组件开启

进行部署之前，请先开启tlqcn组件，并修改镜像repository：使用kafka、amqp、mqtt协议请使用tlq-cn-

base-protocols; 不使用则默认为tlq-cn-base（**注意**：此处只是选择镜像，不会部署这些协议）。

1. 进入tonglinkq-cn目录。

```
cd tonglinkq-cn
```

2. 修改values.yaml中的如下配置（**粗体**为需修改项）：

```
vi values.yaml
```

需修改charts.tlqcn的值为**true**以开启组件：

```
...

charts:

  tlqcn: false

tlqcn:

  images:

    tonglinkqAll:

      repository: tlq-cn-base

...
```

2.3.2. 部署场景说明

注意：根据需要选择部署的规模，也可以先选择最小部署，后面通过修改参数扩容到需要的规格。

2.3.2.1. 最小化部署

最小化部署只适用于功能的验证，不适用于测试和生产环境。不适用于做性能测试。

最小化部署集群配置需求如下：

- 机器配置建议4C 8G及以上；
- 磁盘需求请参考tonglinkq-cn/values.yml中的配置（80Gi以上）：

```
...

tlqcn:

  zookeeper:

    volumes:
```

```
persistence: true
```

```
data:
```

```
name: data
```

```
size: 20Gi
```

```
...
```

```
bookkeeper:
```

```
volumes:
```

```
persistence: true
```

```
journal:
```

```
name: journal
```

```
size: 10Gi
```

```
ledgers:
```

```
name: ledgers
```

```
size: 50Gi
```

```
...
```

2.3.2.2. 一般部署

一般部署适用于小流量场景下的基本功能使用，不适用于做性能测试和生产环境。

Zookeeper，Bookkeeper，Broker副本数均设置为3，Broker的写入模式修改为2:2:2，修改以下配置文件：

1. 进入tonglinkq-cn目录。

```
cd tonglinkq-cn
```

2. 修改values.yaml中的如下配置（粗体为需修改项）：

```
vi values.yaml
```

```
...
```

```
tlqcn:
```



```

zookeeper:

  replicaCount: 3

...

bookkeeper:

  replicaCount: 3

...

broker:

  replicaCount: 3

  configData:

    managedLedgerDefaultEnsembleSize: "2"

    managedLedgerDefaultWriteQuorum: "2"

    managedLedgerDefaultAckQuorum: "2"

...

```

一般部署集群配置需求如下：

- 机器配置建议16C 32G及以上；
- 如需提升组件的资源分配，请根据集群环境情况修改tonglinkq-cn/values.yml中的以下参数配置：

```

...
tlqcn:

  zookeeper:

    resources:

      limits:

        memory: 512Mi

        cpu: 0.2

```

```
    requests:

        memory: 256Mi

        cpu: 0.1
```

...

bookkeeper:

```
    resources:

        limits:

            memory: 1024Mi

            cpu: 0.4

        requests:

            memory: 512Mi

            cpu: 0.2
```

...

broker:

```
    resources:

        limits:

            memory: 1024Mi

            cpu: 1

        requests:

            memory: 512Mi

            cpu: 0.2
```

...

2.3.2.3. 高性能部署

Zookeeper, Bookkeeper, Broker副本数配置大于3, Broker的写入模式可根据bookkeeper的副本数进行配置, broker和bookie的cpu请配置为4, zookeeper的cpu配置为1即可。

高性能部署集群配置需求如下:

- 机器配置建议32C 64G及以上;
- 如需提升组件的资源分配, 请参考[一般部署](#)进行合理配置;
- 根据集群资源配置, 将tonglinkq-cn/values.yml中以下组件的jvm参数配置适量增大:

```
...

tlqcn:

  zookeeper:

    configData:

      PULSAR_MEM: >

      -Xms256m -Xmx256m
    ...

  bookkeeper:

    configData:

      BOOKIE_MEM: >

      -Xms2g

      -Xmx2g

      -XX:MaxDirectMemorySize=2g
    ...

  broker:

    configData:

      PULSAR_MEM: >

      -Xms2g
```

```
-Xmx2g

-XX:MaxDirectMemorySize=4g

...
```

2.3.3. (可选) 配置证书认证

注意：如使用POC模式，则无需配置本章节证书认证。

1. 进入tonglinkq-cn目录。

```
cd tonglinkq-cn
```

2. 修改values.yaml中的如下配置（粗体为需修改项）：

```
vi values.yaml
```

```
...

tlqcn:

  license:

    ips: "10.10.81.175:8889"

    publicKey: "TongLINK/Q-CN对应License的公钥"

...
```

- **license.ips:** 修改该配置的值对应License Server的服务器IP地址和端口。
- **license.publicKey:** 修改该配置的值对应License Server的公钥，可以从License Server 管理控制台获取，详细介绍请参考《License Server 产品用户指南》中“授权证书管理”章节。

2.3.4. 创建命名空间

在K8s服务器的master节点上使用如下命令创建TongLINK/Q-CN部署的命名空间：

```
kubectl create namespace tlq-cn
```

```
[xxx@k8s-master ~]$ kubectl create namespace tlq-cn

namespace/ tlq-cn created
```

其中**tlq-cn**为命名空间名称，用户可自行定义。

2.3.5. helm部署集群

在tonglinkq-cn目录执行如下命令部署TongLINK/Q-CN:

helm install test -n tlq-cn ./tonglinkq-cn

其中test为helm部署的release名称；-n为命名空间名称。

```
[root@k8s-node-136 chart]# helm install test -n tlq-cn ./
NAME: test
LAST DEPLOYED: Fri Sep 27 15:49:02 2024
NAMESPACE: tlq-cn
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
TongLINK/Q-CN 实例部署成功!

集群名称: test
命名空间: tlq-cn

查看TongLINK/Q-CN实例POD运行状况:
kubectl -n tlq-cn get pods

查看服务端口:
kubectl -n tlq-cn get svc test-tonglinkq-cn-proxy
```

说明:

- 若后续需要更新values.yaml文件，使用如下命令更新:

helm upgrade test -n tlq-cn ./tonglinkq-cn

- 如需删除集群，使用如下命令:

helm uninstall test -n tlq-cn

2.3.6. 验证集群部署

在k8s服务器的master节点上执行如下命令查看集群各组件运行状态:

kubectl get pods -n tlq-cn

其中-n为命名空间名称。

如下图所示，结果中有2个为completed状态，其余都为running状态，则集群部署成功。

```
[root@k8s-node-136 chart]# kubectl -n tlq-cn get pods
NAME                                READY   STATUS    RESTARTS   AGE
test-tlqcn-bookie-0                1/1     Running   0           2m14s
test-tlqcn-bookie-init-qptx9       0/1     Completed 0           2m14s
test-tlqcn-broker-0                1/1     Running   0           2m14s
test-tlqcn-cluster-init-wktff      0/1     Completed 0           2m14s
test-tlqcn-proxy-0                 1/1     Running   0           2m14s
test-tlqcn-toolset-0               1/1     Running   0           2m14s
test-tlqcn-zookeeper-0             1/1     Running   0           2m14s
```

2.4. 方法二：Operator部署

2.4.1. 部署operator manager和crds

1. 进入tlqcn-operator目录。

cd tlqcn-operator

2. 使用如下命令部署operator和安装crds。

kubectl apply -f ./

```
> kubectl apply -f ./
namespace/tlqcn-operator-system unchanged
serviceaccount/tlqcn-operator-controller-manager unchanged
role.rbac.authorization.k8s.io/tlqcn-operator-leader-election-role unchanged
clusterrole.rbac.authorization.k8s.io/tlqcn-operator-manager-role unchanged
clusterrole.rbac.authorization.k8s.io/tlqcn-operator-metrics-reader unchanged
clusterrole.rbac.authorization.k8s.io/tlqcn-operator-proxy-role unchanged
rolebinding.rbac.authorization.k8s.io/tlqcn-operator-leader-election-rolebinding unchanged
clusterrolebinding.rbac.authorization.k8s.io/tlqcn-operator-manager-rolebinding unchanged
clusterrolebinding.rbac.authorization.k8s.io/tlqcn-operator-proxy-rolebinding unchanged
service/tlqcn-operator-controller-manager-metrics-service unchanged
deployment.apps/tlqcn-operator-controller-manager configured
customresourcedefinition.apiextensions.k8s.io/bookkeepers.tlqcn.tongtech.com configured
customresourcedefinition.apiextensions.k8s.io/brokers.tlqcn.tongtech.com configured
customresourcedefinition.apiextensions.k8s.io/messagetraces.tlqcn.tongtech.com configured
customresourcedefinition.apiextensions.k8s.io/proxies.tlqcn.tongtech.com configured
customresourcedefinition.apiextensions.k8s.io/zookeepers.tlqcn.tongtech.com configured
```

2.4.2. operator部署集群

1. 进入tlqcn-operator-instance目录。

cd tlqcn-operator-instance

2. 运行如下命令创建命名空间。

kubectl create ns tlqcn-10

其中**tlqcn-10**为命名空间名称，用户可自行定义。

3. 执行如下命令部署TongLINK/Q-CN：

kubectl apply -f ./ -n tlqcn-10

```
> kubectl create ns tlqcn-10
namespace/tlqcn-10 created
> kubectl apply -f ./ -n tlqcn-10
bookkeeper.tlqcn.tongtech.com/tlqcn-operator created
broker.tlqcn.tongtech.com/tlqcn-operator created
messagetrace.tlqcn.tongtech.com/tlqcn-operator created
proxy.tlqcn.tongtech.com/tlqcn-operator created
zookeeper.tlqcn.tongtech.com/tlqcn-operator created
```

4. 如需要更新，可以编辑对应的自定义资源。

a. 自定义资源如下：

- broker资源：brokers.tlqcn.tongtech.com
- bookie资源：bookkeepers.tlqcn.tongtech.com
- zk资源：zookeepers.tlqcn.tongtech.com
- proxy资源：proxies.tlqcn.tongtech.com
- 消息轨迹资源：messagetraces.tlqcn.tongtech.com

b. 查询自定义资源，以查询broker资源为例：

kubectl -n tlqcn-10 get brokers.tlqcn.tongtech.com

```
> kubectl -n tlqcn-10 get brokers.tlqcn.tongtech.com
NAME                AGE
tlqcn-operator      57s
```

c. 编辑自定义资源，以编译broker资源为例：

kubectl -n tlqcn-10 edit brokers.tlqcn.tongtech.com tlqcn-operator

2.4.3. 验证集群部署

在k8s服务器的master节点上执行如下命令查看集群各组件运行状态：

kubectl get pods -n tlq-cn

其中-n为命名空间名称。

如下图所示，结果中有1个为completed状态，其余都为running状态，则集群部署成功。

NAME	READY	STATUS	RESTARTS	AGE
tlqcn-operator-bookkeeper-0	1/1	Running	0	37d
tlqcn-operator-broker-0	1/1	Running	0	37d
tlqcn-operator-cluster-init-0-z8x85	0/1	Completed	0	37d
tlqcn-operator-message-trace-0	1/1	Running	0	37d
tlqcn-operator-proxy-0	1/1	Running	0	37d
tlqcn-operator-zookeeper-0	1/1	Running	0	3m19s

2.5. 业务验证

进入容器验证：

1. 使用如下命令进入toolset容器：

kubect exec -it test-tonglinkq-toolset-0 -n tlq-cn /bin/bash

参数说明：

- **-it**：用户自定义的toolset容器名称；
- **-n**：为命名空间名称。

2. 成功进入容器后执行如下命令：

- **./bin/tong-admin tenants list** #有tenant输出表示成功

```
I have no name!@test-tonglinkq-toolset-0:/pulsar$ ./bin/tong-admin tenants list
Warning: Nashorn engine is planned to be removed from a future JDK release
public
pulsar
I have no name!@test-tonglinkq-toolset-0:/pulsar$
```

- **./bin/tong-client produce my-topic -m sss -n 5** #显示成功生产5条消息表示成功。

```
I have no name!@test-tonglinkq-toolset-0:/pulsar$ ./bin/tong-client produce my-topic -m sss -n 5
Warning: Nashorn engine is planned to be removed from a future JDK release
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.apache.pulsar.common.util.netty.DnsResolverUtil (file:/pulsar/lib/org.apache.pulsar-pulsar-common-2.10.1.jar) to method sun.net.Inet
AddressCachePolicy.get()
WARNING: Please consider reporting this to the maintainers of org.apache.pulsar.common.util.netty.DnsResolverUtil
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
07:43:24.592 [pulsar-client-io-1-1] INFO org.apache.pulsar.client.impl.ConnectionPool - [[id: 0xf37059ac, L:/10.244.2.209:39120 - R:test-tonglinkq-proxy.tlq10.svc.cluster.l
ocal/10.101.156.250:6650]] connected to server
07:43:27.782 [pulsar-client-io-1-1] INFO org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - Starting Pulsar producer perf with config: {"topicName":"my-topic","produ
cerName":null,"sendTimeoutMs":30000,"blockIfQueueFull":false,"maxPendingMessages":1000,"maxPendingMessagesAcrossPartitions":50000,"messageRoutingMode":"RoundRobinPartition",
"hashingScheme":"JavaStringHash","cryptoFailureAction":"FAIL","batchingMaxPublishDelayMicros":1000,"batchingPartitionsSwitchFrequencyByPublishDelay":10,"batchingMaxMessages":
1000,"batchingMaxBytes":131072,"batchingEnabled":true,"chunkingEnabled":false,"compressionType":"NONE","initialSequenceId":null,"autoUpdatePartitions":true,"autoUpdateParti
tionsIntervalSeconds":60,"multiSchema":true,"accessMode":"Shared","lazyStartPartitionedProducers":false,"properties":{},"initialSubscriptionName":null}
07:43:28.280 [pulsar-client-io-1-1] INFO org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - Pulsar client config: {"serviceUrl":"pulsar://test-tonglinkq-proxy:6650",
"authPluginClassName":null,"authParams":null,"authParamMap":null,"operationTimeoutMs":30000,"lookupTimeoutMs":30000,"statsIntervalSeconds":60,"numIoThreads":1,"numListenerTh
reads":1,"connectionsPerBroker":1,"useTcpNodeLAY":true,"useTls":false,"tlsTrustCertsFilePath":"","tlsAllowInsecureConnection":false,"tlsHostNameVerificationEnabled":false,"co
ncurrentLookupRequest":5000,"maxLookupRequest":50000,"maxLookupRedirects":20,"maxNumberOfRejectedRequestPerConnection":50,"keepAliveIntervalSeconds":30,"connectionTimeoutMs":
10000,"requestTimeoutMs":60000,"initialBackoffIntervalNanos":100000000,"maxBackoffIntervalNanos":60000000000,"enableBusyWait":false,"listenerName":null,"useKeyStoreTls":fal
se,"sslProvider":null,"tlsTrustStoreType":"JKS","tlsTrustStorePath":"","tlsTrustStorePassword":"","tlsCipherSuites":["TLSv1","TLSv1.1","TLSv1.2","TLSv1.3"],"tlsProtocols":[],"memoryLimitBytes":10,"proxyServiceUR
l":null,"proxyProtocol":null,"enableTransaction":false,"dnsLookupBindAddress":null,"dnsLookupBindPort":0,"socks5ProxyAddress":null,"socks5ProxyUsername":null,"socks5ProxyPas
sword":null}
07:43:28.383 [pulsar-client-io-1-1] INFO org.apache.pulsar.client.impl.ConnectionPool - [[id: 0x31b7bb88, L:/10.244.2.209:39132 - R:test-tonglinkq-proxy.tlq10.svc.cluster.l
ocal/10.101.156.250:6650]] connected to server
07:43:28.477 [pulsar-client-io-1-1] INFO org.apache.pulsar.client.impl.ClientCnx - [id: 0x31b7bb88, L:/10.244.2.209:39132 - R:test-tonglinkq-proxy.tlq10.svc.cluster.local/1
0.101.156.250:6650] Connected through proxy to target broker at test-tonglinkq-broker-0.test-tonglinkq-broker.tlq10.svc.cluster.local:6650
07:43:28.577 [pulsar-client-io-1-1] INFO org.apache.pulsar.client.impl.ProducerImpl - [my-topic] [null] Creating producer on cnx [id: 0x31b7bb88, L:/10.244.2.209:39132 - R:
test-tonglinkq-proxy.tlq10.svc.cluster.local/10.101.156.250:6650]
07:43:28.876 [pulsar-client-io-1-1] INFO org.apache.pulsar.client.impl.ProducerImpl - [my-topic] [test-tonglinkq-1-4] Created producer on cnx [id: 0x31b7bb88, L:/10.244.2.2
09:39132 - R:test-tonglinkq-proxy.tlq10.svc.cluster.local/10.101.156.250:6650]
07:43:29.181 [main] INFO com.scurellous.circe.checksum.Crc32CIntChecksum - SSE4.2 CRC32C provider initialized
07:43:29.396 [main] INFO org.apache.pulsar.client.impl.ProducerImpl - client closing. URL: pulsar://test-tonglinkq-proxy:6650
07:43:29.478 [main] INFO org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - [my-topic] [test-tonglinkq-1-4] Pending messages: 0 --- Publish throughput: 4.48 msg/s ---
0.00 Msg/s --- Latency: med: 8.000 ms - 95pct: 311.000 ms - 99pct: 311.000 ms - max: 311.000 ms --- BatchSize: med: 1.000 - 95pct: 1.000 - 99pct: 1.
000 - 99.9pct: 1.000 - max: 1.000 --- MsgSize: med: 3.000 bytes - 95pct: 3.000 bytes - 99pct: 3.000 bytes - max: 3.000 bytes --- Ack received rate: 4.
48 ack/s --- Failed messages: 0 --- Pending messages: 0
07:43:29.487 [pulsar-client-io-1-1] INFO org.apache.pulsar.client.impl.ProducerImpl - [my-topic] [test-tonglinkq-1-4] Closed Producer
07:43:29.581 [pulsar-client-io-1-1] INFO org.apache.pulsar.client.impl.ClientCnx - [id: 0xf37059ac, L:/10.244.2.209:39120 - R:test-tonglinkq-proxy.tlq10.svc.cluster.local/1
0.101.156.250:6650] Disconnected
07:43:29.585 [pulsar-client-io-1-1] INFO org.apache.pulsar.client.impl.ClientCnx - [id: 0x31b7bb88, L:/10.244.2.209:39132 - R:test-tonglinkq-proxy.tlq10.svc.cluster.local/1
0.101.156.250:6650] Disconnected
07:43:31.601 [main] INFO org.apache.pulsar.client.cli.PulsarClientTool - 5 messages successfully produced
I have no name!@test-tonglinkq-toolset-0:/pulsar$
```

参数说明：

- **-m**：生产消息的消息体内容。
- **-n**：生产消息的数量。

3. 虚拟机部署

在虚拟机上部署TongLINK/Q-CN集群包括包括“单机部署”（参见[单机部署](#)章节）和“集群部署”（参见[集群部署](#)章节）。

3.1. 部署前准备

3.1.1. 运行环境

TongLINK/Q-CN需运行在64位Linux系统环境下。系统要求满足以下条件：

系统组件	系统要求
硬件	Linux操作系统所支持的硬件平台。
操作系统	Linux
JAVA环境	必须使用JRE/JDK17及以上版本

要在裸机上运行TongLINK/Q-CN，根据不同的使用场景（处理消息量的大小不同），推荐使用以下两种部署方式：

- 单机版集群部署，仅用于开发进行测试，请参考[单机部署](#)章节。
 - 1台机器，仅用于测试环境及基本生产、消费的场景。
 - 启动Standlaone单机版
 - 机器建议配置不低于4C 8G
- 一般性能集群部署，请参考[集群部署](#)章节。
 - 推荐3台机器，后续可以扩容
 - 每台机器都部署1个Zookeeper，1个Bookkeeper和1个Broker；混合部署需要注意服务间的端口冲突
 - 每台机器建议配置不低于8C 16G
 - 涵盖所有broker hosts的单个DNS名称。（可选）

3.1.2. 获取安装包

部署TongLINK/Q-CN需要获取的安装包如下：

安装包名称	说明	解压后目录名
install_TLQ-CN-ALL_10.0.x.x.tar.gz	Linux环境下TongLINK/Q-CN的安装包，不使用kafka、amqp、mqtt协议时，请下载此包。	TLQ-CN

安装包名称	说明	解压后目录名
install_TLQ-CN-ALL-PROTOCOLS_10.0.x.x.tar.gz	Linux环境下TongLINK/Q-CN的安装包，使用kafka、amqp、mqtt协议时，请下载此包。	TLQ-CN
jdk-17_linux-x64_bin.tar.gz	X86对应的JDK17软件包（根据使用CPU类型选择）。	-
jdk-17_linux-aarch64_bin.tar.gz	ARM对应的JDK17软件包（根据使用CPU类型选择）。	-
license Server软件安装包（可选）	用于安装license Server。	-

注意：请在官网下载JDK：<https://www.oracle.com/java/technologies/javase/jdk17-0-13-later-archive-downloads.html>

3.1.3. 上传并解压安装包

1. 根据是否使用kafka、amqp、mqtt协议获取对应的产品安装包，将产品安装包上传到需要部署的Linux服务器上（Standalone部署上传1台服务器即可，3节点集群部署需上传3台服务器），如Linux服务器的/home目录下。
2. 进入/home目录，执行解压命令：

```
tar zxvf install_TLQ-CN-ALL_10.0.x.x.tar.gz
```

或

```
tar zxvf install_TLQ-CN-ALL-PROTOCOLS_10.0.x.x.tar.gz
```

3.1.4. 目录说明

TongLINK/Q-CN的安装包解压后，/home/TLQ-CN目录下的子目录及文件如下。

目录名称	说明
bin	包含所有的CLI工具，包括tong-admin, tong-client, bookkeeper, tong等。
conf	包含所有的配置文件，broker配置，zookeeper配置等。
examples	包含function使用的示例文件。
instances	包含使用function的java实例和python实例。
lib	TongLINK/Q-CN运行所需的jar包。
protocols	包含kafka、amqp、mqtt插件。

目录名称	说明
interceptors	包含消息轨迹、消息过滤插件。

3.1.5. 配置JDK

TongLINK/Q-CN支持以下两种方式设置JDK，请用户根据实际需求选择合适的方式进行设置：可以按照自己的方式设置JDK的环境变量（方法一），也可以按照配置JDK路径的方式（方法二），二选一即可。

- 方法一：自定义设置JDK环境变量

执行命令java -version查询JDK版本，系统应输出JDK17或更高版本，示例如下：

```
-> # java -version
java version "17.0.9" 2023-10-17 LTS
Java(TM) SE Runtime Environment (build 17.0.9+11-LTS-201)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.9+11-LTS-201, mixed mode, sharing)
```

- 方法二：通过配置文件设置JDK路径

1. 使用如下命令解压JDK包：

```
tar zxvf jdk-17_linux-x64_bin.tar.gz
```

解压完成后，JDK文件将存放于指定路径下，例如：/usr/local/java/jdk-17.0.7

2. 配置JDK路径：

- a. 在TLQ-CN路径下编辑bin/tong文件：

```
vim bin/tong
```

- b. 找到以下行：

```
# Set JAVA_HOME here to override the environment setting
# JAVA_HOME=
```

- c. 设置JAVA_HOME为真实的JDK路径，同时取消注释（去掉前面的“#”）：

```
# Set JAVA_HOME here to override the environment setting
JAVA_HOME=/usr/local/java/jdk-17.0.7
```

- d. 参考以上步骤a ~ c修改conf/pulsar_tools_env.sh文件并设置JAVA_HOME为真实的JDK路径。

- e. 参考以上步骤a ~ c修改conf/pulsar_env.sh文件并设置JAVA_HOME为真实的JDK路径。

注意：如果部署的是3节点集群，则3台服务器上都需要进行上述配置。

3.1.6. 确定集群名称

确定待部署的集群名称，该名称将在后续监控接入和控制台管理中使用。建议命名如下：

tlq-<机房位置>-<业务名称缩写>-<用途>

例如：位于北京机房，处理日志的测试环境，则集群可以命名为：

tlq-bj-log-test

确定的集群名称将在以下配置中使用：

- [修改Standalone配置文件](#)章节的**clusterName**参数。
- [初始化元数据](#)章节的**cluster**参数。
- [修改Broker配置文件](#)章节的**clusterName**参数。

3.1.7. （可选）安装License Server

安装License Server 4.5.1.6及以上版本，详细安装步骤请参考《License Server 安装部署指南》。

注意：

- 若已经安装东方通其他产品，可以共用其他产品的License Server。
- License Server可以在安装TongLINK/Q-CN后续2个月内安装，POC阶段可暂时不安装。如需安装，请联系厂家支持。

3.2. 单机部署

3.2.1. （可选）修改Standalone配置文件

1. 进入服务器的/home/TLQ-CN目录。

```
cd /home/TLQ-CN
```

2. 修改standalone配置文件。

```
vim conf/standalone
```

修改如下参数：

```
.....
# license server valid

licenseIps=

licensePublicKey=

# Name of the cluster to which this broker belongs to

clusterName=<clusterName>
.....
```

参数说明如下表所示：

参数	描述
clusterName	集群名称。参见 确定集群名称 章节相关介绍，也可以使用默认名称。
licenseIps	License Server服务器IP地址和端口。POC测试可不配置。
licensePublicKey	TongLINK/Q-CN产品的公钥，可以从License Server管理控制台获取，详细介绍请参考《License Server 产品用户指南》中“授权证书管理”章节。POC测试可不配置。

3. 保存配置文件并退出。

3.2.2. 启停Standalone服务

- 启动Standalone服务

在所在服务器的/home/TLQ-CN目录运行以下命令启动Standalone服务：

- 前台启动：

bin/tong standalone

出现如下打印表示启动成功：

```
2025-03-28T14:47:39,517+0800 [worker-scheduler-0] INFO
org.apache.pulsar.functions.worker.SchedulerManager - Schedule summary - execution time:
0.048590289 sec | total unassigned: 0 | stats: {"Added": 0, "Updated": 0, "removed": 0}

{
  "c-standalone-fw-10.10.86.248-8080" : {
    "originalNumAssignments" : 0,
    "finalNumAssignments" : 0,
    "instancesAdded" : 0,
    "instancesRemoved" : 0,
    "instancesUpdated" : 0,
    "alive" : true
  }
}
```

- 后台启动:

bin/tong-daemon start standalone

查看端口6650和8080是否启动成功:

```
[root@k8s-node1 ~]# netstat -lntp |grep -E '6650|8080'
tcp        0      0 0.0.0.0:6650        0.0.0.0:*          LISTEN     19617/java
tcp        0      0 0.0.0.0:8080        0.0.0.0:*          LISTEN     19617/java
[root@k8s-node1 ~]#
```

- 停止Standalone服务
 - 以前台方式启动后，在启动程序运行窗口直接按下Ctrl+C即可停止Standalone服务。
 - 以后台方式启动后，在/home/TLQ-CN目录运行以下命令停止Standalone服务。

bin/tong-daemon stop standalone

3.3. 集群部署

说明：以下部署流程，以使用3台机器部署一般性能集群为例。

3.3.1. 部署Zookeeper集群

3.3.1.1. 修改Zookeeper配置文件

1. 分别进入3台服务器的/home/TLQ-CN目录。

cd /home/TLQ-CN

2. 修改zookeeper.conf配置文件。

vi conf/zookeeper.conf

3个节点上的配置文件都需要修改，具体配置如下:

- a. 新增如下配置:

```
server.1=node1 ip:port1:port2

server.2=node2 ip:port1:port2

server.3=node3 ip:port1:port2
```

示例如下:

```
server.1=10.10.86.247:2888:3888

server.2=10.10.86.248:2888:3888
```

```
server.3=10.10.86.249:2888:3888
```

参数说明：

server.1~n：为Zookeeper集群的各节点指定编号。3.5版本后格式规范为：

server.<positive id> = <address1>:<port1>:<port2>

- address1为当前Zookeeper节点所在服务器的IP；
- port1和port2两个端口可自定义，一般为2888和3888；

b. 修改如下参数：

```
metricsProvider.httpPort=自定义端口号，建议使用8001，不要使用8000（会跟bookie端口冲突）
```

3. 新建相关文件夹。

分别在3个Zookeeper节点所在服务器上，新建data/zookeeper目录。

mkdir -p data/zookeeper

4. 创建节点id。

为每个Zookeeper节点新建myid文件。在对应的节点执行对应的命令，例如在server.1上执行

echo 1 >data/zookeeper/myid。

节点1: echo 1 >data/zookeeper/myid

节点2: echo 2 >data/zookeeper/myid

节点3: echo 3 >data/zookeeper/myid

...

节点n: echo n >data/zookeeper/myid

3.3.1.2. 启停Zookeeper

• 启动Zookeeper节点

◦ 分别在3个节点所在服务器的/home/TLQ-CN目录下运行以下命令启动Zookeeper节点：

- 前台启动：

bin/tong zookeeper

注意：第一个Zookeeper节点启动后会去找第二个Zookeeper节点的地址进行选举，因为第二个Zookeeper节点还未启动，所以会报连接错误，再启动第二个Zookeeper节点即可。

- 后台启动：

bin/tong-daemon start zookeeper

```
[xxx@master1 TLQ-CN]# bin/tong-daemon start zookeeper
```

```
doing start zookeeper ...
```

```
starting zookeeper, logging to /home/TLQ-CN/logs/pulsar-zookeeper-k8s-worker1.log
```

Note: Set immediateFlush to true in conf/log4j2.yaml will guarantee the logging event is flushing to disk immediately. The default behavior is switched off due to performance considerations.

启动日志中输出starting zookeeper, logging to /home/TLQ-CN/logs/pulsar-zookeeper-k8s-worker1.log。其中日志输出到/home/TLQ-CN/logs/pulsar-zookeeper-k8s-worker1.log文件中，3个节点输出的日志文件命名有所不同，日志文件命名规则为：pulsar-zookeeper-{hostname}.log，可在该文件中查看日志。

- 在任意节点上的/home/TLQ-CN目录下使用如下命令检查Zookeeper节点是否已经启动：

bin/tong zookeeper-shell

```
2023-06-14T14:15:58,036+0800 [main] INFO org.apache.zookeeper.Cli
2023-06-14T14:15:58,043+0800 [main] INFO org.apache.zookeeper.Cli
2023-06-14T14:15:58,053+0800 [main] INFO org.apache.zookeeper.Cli
Welcome to ZooKeeper!
2023-06-14T14:15:58,133+0800 [main-SendThread(localhost:2181)] IN
2023-06-14T14:15:58,134+0800 [main-SendThread(localhost:2181)] IN
2023-06-14T14:15:58,142+0800 [main-SendThread(localhost:2181)] IN
r: localhost/127.0.0.1:2181
2023-06-14T14:15:58,170+0800 [main-SendThread(localhost:2181)] IN
10023c9f58d0000, negotiated timeout = 30000

WATCHER::

WatchedEvent state:SyncConnected type:None path:null
JLine support is enabled
[zk: localhost:2181(CONNECTED) 0] █
```

出现如上图所示的信息则说明Zookeeper已经正常启动。

- 也可以通过查看Zookeeper的端口判断是否启动：

```
netstat -lntp | grep 2181
```

• (可选) 停止Zookeeper节点

- 以前台方式启动，分别在3个节点启动程序运行窗口直接按下Ctrl+C即可停止Zookeeper节点。
- 以后台方式启动，分别在3个节点所在机器的/home/TLQ-CN目录下运行以下命令停止Zookeeper节点：

bin/tong-daemon stop zookeeper

```
[xxx@master1 TLQ-CN]# bin/tong-daemon stop zookeeper
```

```
doing stop zookeeper ...
```



```
stopping zookeeper

Shutdown is in progress... Please wait...

Shutdown completed.
```

3.3.2. 初始化元数据

在任意一台已部署Zookeeper的服务器上，进入/home/TLQ-CN目录，执行以下命令。需将其中的zk1，zk2，zk3，broker-1-ip,broker-2-ip,broker-3-ip更换为实际使用的IP地址。

初始化元数据：

```
bin/tong initialize-cluster-metadata \

--cluster <cluster-Name> \

--metadata-store zk:zk1:2181,zk2:2181,zk3:2181 \

--configuration-metadata-store zk:zk1:2181,zk2:2181,zk3:2181 \

--web-service-url http://broker1-ip:port,broker2-ip:port,broker3-ip:port \

--broker-service-url pulsar://broker1-ip:port,broker2-ip:port,broker3-ip:port
```

使用示例如下：

```
bin/tong initialize-cluster-metadata \

--cluster tlq-bj-log-test \

--metadata-store zk:10.10.86.247:2181,10.10.86.248:2181,10.10.86.249:2181 \

--configuration-metadata-store zk:10.10.86.247:2181,10.10.86.248:2181,10.10.86.249:2181 \

--web-service-url http://10.10.86.247:8080,10.10.86.248:8080,10.10.86.249:8080 \

--broker-service-url pulsar://10.10.86.247:6650,10.10.86.248:6650,10.10.86.249:6650
```

参数说明如下表所示：

参数	描述
--cluster	集群名称，参见 确定集群名称 章节相关介绍。
--metadata-store	集群的“本地”元数据存储连接字符串。

参数	描述
--configuration-metadata-store	配置元数据存储整个实例的连接字符串。
--web-service-url	集群的web服务URL和端口。此URL应该是标准DNS名称，默认端口是8080（最好不要使用其他端口）。
--broker-service-url	broker服务URL，用于与集群中的broker进行交互。此URL需要使用与web服务URL不同的DNS名称，默认端口为6650（最好不要使用其他端口）

如果没有DNS服务器，则可以在具有以下设置的服务URL中使用多主机格式：

```
--web-service-url http://host1:8080,host2:8080,host3:8080 \
--broker-service-url pulsar://host1:6650,host2:6650,host3:6650 \
```

3.3.3. 部署BookKeeper集群

3.3.3.1. 修改BookKeeper配置文件

1. 分别进入3台服务器的/home/TLQ-CN目录。

```
cd /home/TLQ-CN
```

2. 修改bookkeeper.conf配置文件。

```
vi conf/bookkeeper.conf
```

3个节点上的配置文件都需要修改，具体配置如下：

```
metadataServiceUri=zk://zk1:2181;zk2:2181;zk3:2181/ledgers

advertisedAddress=本机ip地址

prometheusStatsHttpPort=7000

licenseIps=License Server服务器ip:端口

licensePublicKey=产品public key
```

配置示例如下：

```
metadataServiceUri=zk://10.10.86.247:2181;10.10.86.248:2181;10.10.86.249:2181/ledgers

advertisedAddress=本机ip地址
```

```
prometheusStatsHttpPort=7000
```

```
licenseIps=10.10.81.174:8888
```

```
licensePublicKey=
```

参数说明如下表所示：

参数	说明
metadataServiceUri	指定本地Zookeeper集群，用来将BookKeeper节点的元数据存放在Zookeeper集群。
advertisedAddress	本机ip地址。
prometheusStatsHttpPort	普罗米修斯端口号，默认8000，跟BookKeeper的httpServerPort端口冲突，所以需要修改，示例修改为7000。
licenseIps（可选）	License Server服务器IP地址和端口。POC测试可不配置。
licensePublicKey（可选）	TongLINK/Q-CN产品的公钥，可以从License Server管理控制台获取，详细介绍请参考《License Server 产品用户指南》中“授权证书管理”章节。POC测试可不配置。

3. 初始化元数据。

在3个节点中的任意一个节点的/home/TLQ-CN目录下执行**bin/bookkeeper shell metaformat**初始化元数据，遇到提示按“y”，只需执行一次。

```
[xxx@master1 TLQ-CN]# bin/bookkeeper shell metaformat
```

```
JAVA_HOME not set, using java from PATH. (/usr/bin/java)
```

```
2023-05-19T18:04:33,095+0800 [main] INFO org.apache.bookkeeper.meta.MetadataDrivers - BookKeeper metadata driver manager initialized
```

```
2023-05-19T18:04:33,109+0800 [main] INFO org.apache.bookkeeper.meta.zk.ZKMetadataDriverBase - Initialize zookeeper metadata driver at metadata service uri zk+null://10.10.86.247:2181; 10.10.86.248:2181; 10.10.86.249:2181/ledgers : zkServers = 10.10.86.247:2181, 10.10.86.248:2181, 10.10.86.249:2181, ledgersRootPath = /ledgers.
```

```
.....
```

4. 输出以下结果：

```
2023-06-25T18:41:09.071+0800 [main] INFO org.apache.zookeeper.ZooKeeper - Initiating client connection, connectString=10.10.86.247:2181,10.10.86.248:2181,10.10.86.249:2181 sessionTimeo
ut=30000 watcher=org.apache.bookkeeper.zookeeper.ZooKeeperWatcherBase@2484f433
2023-06-25T18:41:09.078+0800 [main] INFO org.apache.zookeeper.common.XS09Util - Setting -Djdk.tls.rejectClientInitiatedRenegotiation=true to disable client-initiated TLS renegotiation
2023-06-25T18:41:09.085+0800 [main] INFO org.apache.zookeeper.ClientCnxnSocket - jute.maxbuffer value is 1048575 Bytes
2023-06-25T18:41:09.097+0800 [main] INFO org.apache.zookeeper.ClientCnxn - zookeeper.request.timeout value is 0. feature enabled=false
2023-06-25T18:41:09.912+0800 [main-SendThread[10.10.86.248:2181]] INFO org.apache.zookeeper.ClientCnxn - Opening socket connection to server k8s-node1/10.10.86.248:2181.
2023-06-25T18:41:09.912+0800 [main-SendThread[10.10.86.248:2181]] INFO org.apache.zookeeper.ClientCnxn - SASL config status: Will not attempt to authenticate using SASL (unknown error)
2023-06-25T18:41:09.921+0800 [main-SendThread[10.10.86.248:2181]] INFO org.apache.zookeeper.ClientCnxn - Socket connection established, initiating session, client: /10.10.86.247:40844,
server: k8s-node1/10.10.86.248:2181
2023-06-25T18:41:09.922+0800 [main-SendThread[10.10.86.248:2181]] INFO org.apache.zookeeper.ClientCnxn - Session establishment complete on server k8s-node1/10.10.86.248:2181, session i
d = 0x2001992eee00003, negotiated timeout = 30000
2023-06-25T18:41:09.937+0800 [main-EventThread] INFO org.apache.bookkeeper.zookeeper.ZooKeeperWatcherBase - ZooKeeper client is connected now.
Ledger root already exists. Are you sure to format bookkeeper metadata? This may cause data loss. (Y or N) Y
2023-06-25T18:41:13.977+0800 [main] INFO org.apache.bookkeeper.discover.ZKRegistrationManager - Successfully formatted BookKeeper metadata
2023-06-25T18:41:14.085+0800 [main] INFO org.apache.zookeeper.ZooKeeper - Session: 0x2001992eee00003 closed
2023-06-25T18:41:14.085+0800 [main-EventThread] INFO org.apache.zookeeper.ClientCnxn - EventThread shut down for session: 0x2001992eee00003
[root@k8s-master bk]#
```

3.3.3.2. 启停BookKeeper

• 启动BookKeeper

- 分别在3个节点所在服务器的/home/TLQ-CN目录下运行以下命令启动BookKeeper:

- 前台启动:

bin/tong bookie

- 后台启动:

bin/tong-daemon start bookie

```
[xxx@master1 TLQ-CN]# bin/tong-daemon start bookie
```

doing start bookie ...

starting bookie, logging to /home/TLQ-CN/logs/pulsar-bookie-k8s-worker1.log

Note: Set immediateFlush to true in conf/log4j2.yaml will guarantee the logging event is flushing to disk immediately. The default behavior is switched off due to performance considerations.

启动日志中输出starting bookie, logging to /home/TLQ-CN/logs/pulsar-bookie-k8s-worker1.log。其中日志输出到/home/TLQ-CN/logs/pulsar-bookie-k8s-worker1.log文件中，3个节点输出的日志文件命名有所不同，日志文件命名规则为：pulsar-bookie-{hostname}.log，可在该文件中查看日志。

- 当Bookkeeper服务启动完成后，可以使用以下命令校验服务是否正常：出现“Bookie sanity test succeeded”代表启动成功。分别在3个节点所在服务器的/home/TLQ-CN目录下执行以下命令：

bin/bookkeeper shell bookiesanity

```
2023-06-25T18:51:28.071+0800 [main] INFO org.apache.zookeeper.ZooKeeper - Initiating client connection, connectString=10.10.86.247:2181,10.10.86.248:2181,10.10.86.249:2181 sessionTimeo
ut=30000 watcher=org.apache.bookkeeper.zookeeper.ZooKeeperWatcherBase@48c40605
2023-06-25T18:51:28.078+0800 [main] INFO org.apache.zookeeper.common.XS09Util - Setting -Djdk.tls.rejectClientInitiatedRenegotiation=true to disable client-initiated TLS renegotiation
2023-06-25T18:51:28.084+0800 [main] INFO org.apache.zookeeper.ClientCnxnSocket - jute.maxbuffer value is 1048575 Bytes
2023-06-25T18:51:28.093+0800 [main] INFO org.apache.zookeeper.ClientCnxn - zookeeper.request.timeout value is 0. feature enabled=false
2023-06-25T18:51:28.701+0800 [main-SendThread[10.10.86.248:2181]] INFO org.apache.zookeeper.ClientCnxn - Opening socket connection to server k8s-node1/10.10.86.248:2181.
2023-06-25T18:51:28.701+0800 [main-SendThread[10.10.86.248:2181]] INFO org.apache.zookeeper.ClientCnxn - SASL config status: Will not attempt to authenticate using SASL (unknown error)
2023-06-25T18:51:28.709+0800 [main-SendThread[10.10.86.248:2181]] INFO org.apache.zookeeper.ClientCnxn - Socket connection established, initiating session, client: /10.10.86.247:53184,
server: k8s-node1/10.10.86.248:2181
2023-06-25T18:51:28.722+0800 [main-SendThread[10.10.86.248:2181]] INFO org.apache.zookeeper.ClientCnxn - Session establishment complete on server k8s-node1/10.10.86.248:2181, session i
d = 0x2001992eee00007, negotiated timeout = 30000
2023-06-25T18:51:28.724+0800 [main-EventThread] INFO org.apache.bookkeeper.zookeeper.ZooKeeperWatcherBase - ZooKeeper client is connected now.
2023-06-25T18:51:28.909+0800 [main] INFO org.apache.bookkeeper.client.BookKeeper - Weighted ledger placement is not enabled
2023-06-25T18:51:29.008+0800 [main-EventThread] INFO org.apache.bookkeeper.discover.ZKRegistrationClient - Update BookieInfoCache (writable bookie) 10.10.86.247:3181 -> BookieServiceIn
fo(properties={}, endpoints=[EndpointInfo{id=httpserver, port=8080, host=0.0.0.0, protocol=http, auth=[], extensions=[]}, EndpointInfo{id=bookie, port=3181, host=10.10.86.247, protocol=
bookie-rpc, auth=[], extensions=[]}])
2023-06-25T18:51:29.023+0800 [main-EventThread] INFO org.apache.bookkeeper.discover.ZKRegistrationClient - Update BookieInfoCache (writable bookie) 10.10.86.248:3181 -> BookieServiceIn
fo(properties={}, endpoints=[EndpointInfo{id=httpserver, port=8080, host=0.0.0.0, protocol=http, auth=[], extensions=[]}, EndpointInfo{id=bookie, port=3181, host=10.10.86.248, protocol=
bookie-rpc, auth=[], extensions=[]}])
2023-06-25T18:51:29.147+0800 [main-EventThread] INFO org.apache.bookkeeper.client.LedgerCreateOp - Ensemble: [10.10.86.247:3181] for Ledger: 0
2023-06-25T18:51:29.149+0800 [main] INFO org.apache.bookkeeper.tools.cli.commands.bookie.SanityTestCommand - Create ledger 0
2023-06-25T18:51:29.623+0800 [bookkeeper-10-3-1] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - Successfully connected to bookie: 10.10.86.247:3181 [id: 0x41a1bbfd, L:/10.10
.86.247:33416 - R:10.10.86.247/10.10.86.247:3181]
2023-06-25T18:51:29.636+0800 [bookkeeper-10-3-1] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - connection [id: 0x41a1bbfd, L:/10.10.86.247:33416 - R:10.10.86.247/10.10.86.2
47:3181] authenticated as BookKeeperPrincipal(AQ0W0XWU5)
2023-06-25T18:51:29.726+0800 [main] INFO org.apache.bookkeeper.tools.cli.commands.bookie.SanityTestCommand - Written 10 entries in ledger 0
2023-06-25T18:51:29.780+0800 [BookKeeperClientWorker-OrderedExecutor-0-0] INFO org.apache.bookkeeper.client.ReadOnlyLedgerHandle - Closing recovered ledger 0 at entry 9
2023-06-25T18:51:29.808+0800 [main] INFO org.apache.bookkeeper.tools.cli.commands.bookie.SanityTestCommand - Read 10 entries from ledger 0
2023-06-25T18:51:29.836+0800 [main] INFO org.apache.bookkeeper.tools.cli.commands.bookie.SanityTestCommand - Deleted ledger 0
2023-06-25T18:51:29.836+0800 [main] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - Closing the per channel bookie client for 10.10.86.247:3181
2023-06-25T18:51:29.840+0800 [main] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - Closing the per channel bookie client for 10.10.86.247:3181
2023-06-25T18:51:29.840+0800 [bookkeeper-10-3-1] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - Disconnected from bookie channel [id: 0x41a1bbfd, L:/10.10.86.247:33416 ! R:1
0.10.86.247/10.10.86.247:3181]
2023-06-25T18:51:31.795+0800 [main] WARN org.apache.bookkeeper.client.BookKeeper - The mainWorkerPool did not shutdown cleanly
2023-06-25T18:51:31.906+0800 [main] INFO org.apache.zookeeper.ZooKeeper - Session: 0x2001992eee00007 closed
2023-06-25T18:51:31.906+0800 [main-EventThread] INFO org.apache.zookeeper.ClientCnxn - EventThread shut down for session: 0x2001992eee00007
2023-06-25T18:51:31.906+0800 [main] INFO org.apache.bookkeeper.tools.cli.commands.bookie.SanityTestCommand - Bookie sanity test succeeded
[root@k8s-master bk]#
```

- 也可以通过查看是否有bookkeeper的端口3181和8000校验服务是否正常：

```
netstat -lntp |grep -E '3181 |8000'
```

- **（可选）停止BookKeeper服务**

- 以前台方式启动，分别在3个节点启动程序运行窗口直接按下Ctrl+C即可停止BookKeeper。
- 以后台方式启动，分别在3个节点所在服务器的/home/TLQ-CN目录下运行以下命令停止BookKeeper：

```
bin/tong-daemon stop bookie
```

```
[xxx@master1 TLQ-CN]# bin/tong-daemon stop bookie

doing stop bookie...

stopping bookie

Shutdown is in progress... Please wait...

Shutdown completed.
```

- **验证集群内的BookKeeper是否都能正常工作**

在/home/TLQ-CN目录下执行以下命令：

```
bin/bookkeeper shell simpletest --ensemble <num-bookies> --writeQuorum <num-bookies> --ackQuorum
<num-bookies> --numEntries <num-entries>
```

参数说明如下：

参数	说明
ensemble	BookKeeper集群中的节点总数。
writeQuorum	每次并行写入的节点数量。
ackQuorum	写入成功的节点返回数量。
numEntries	可选参数，写入的数据条数。

例如（3节点情况下）：bin/bookkeeper shell simpletest --ensemble 3 --writeQuorum 3
--ackQuorum 3

终端显示如下结果证明Bookkeeper启动成功。

```

2023-06-25T19:11:59.452+0800 [main-EventThread] INFO org.apache.bookkeeper.discover.ZKRegistrationClient - Update BookieInfoCache (writable bookie) 10.10.86.249:3181 -> BookieServiceIn
fo(properties={}, endpoints=[EndpointInfo{id=hostname, port=8080, host=0.0.0.0, protocol=http, auth=[], extensions=[]}], EndpointInfo{id=bookie, port=3181, host=10.10.86.249, protocol=
bookie-rpc, auth=[], extensions=[]})
2023-06-25T19:11:59.459+0800 [BookKeeperClientScheduler-OrderedScheduler-0-0] WARN org.apache.bookkeeper.client.TopologyAwareEnsemblePlacementPolicy - Failed to resolve network locatio
n for 10.10.86.247, using default rack for it : /default-rack.
2023-06-25T19:11:59.461+0800 [BookKeeperClientScheduler-OrderedScheduler-0-0] INFO org.apache.bookkeeper.net.NetworkTopologyImpl - Adding a new node: /default-rack/10.10.86.247:3181
2023-06-25T19:11:59.462+0800 [BookKeeperClientScheduler-OrderedScheduler-0-0] WARN org.apache.bookkeeper.client.TopologyAwareEnsemblePlacementPolicy - Failed to resolve network locatio
n for 10.10.86.248, using default rack for it : /default-rack.
2023-06-25T19:11:59.463+0800 [BookKeeperClientScheduler-OrderedScheduler-0-0] INFO org.apache.bookkeeper.net.NetworkTopologyImpl - Adding a new node: /default-rack/10.10.86.248:3181
2023-06-25T19:11:59.491+0800 [main] WARN org.apache.bookkeeper.client.BookieWatcherImpl - New ensemble: [10.10.86.249:3181, 10.10.86.247:3181, 10.10.86.248:3181] is not adhering to Pla
cement Policy: quarantinedBookies: []
2023-06-25T19:11:59.559+0800 [main-EventThread] INFO org.apache.bookkeeper.client.LedgerCreateOp - Ensemble: [10.10.86.249:3181, 10.10.86.247:3181, 10.10.86.248:3181] for Ledger: 7
2023-06-25T19:11:59.562+0800 [main] INFO org.apache.bookkeeper.tools.cli.commands.client.SimpleTestCommand - Ledger ID: 7
2023-06-25T19:11:59.682+0800 [bookkeeper-10-3-2] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - Successfully connected to bookie: 10.10.86.247:3181 [id: 0xcce08be7a, L:/10.10.
86.247:48294 - R:10.10.86.247/10.10.86.247:3181]
2023-06-25T19:11:59.682+0800 [bookkeeper-10-3-1] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - Successfully connected to bookie: 10.10.86.249:3181 [id: 0xida31afc, L:/10.10.
86.247:51494 - R:10.10.86.249/10.10.86.249:3181]
2023-06-25T19:11:59.682+0800 [bookkeeper-10-3-2] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - Successfully connected to bookie: 10.10.86.248:3181 [id: 0x62b69d34, L:/10.10.
86.247:39422 - R:10.10.86.248/10.10.86.248:3181]
2023-06-25T19:11:59.695+0800 [bookkeeper-10-3-2] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - connection [id: 0xcce08be7a, L:/10.10.86.247:48294 - R:10.10.86.247/10.10.86.2
47:3181] authenticated as BookKeeperPrincipal[ANONYMOUS]
2023-06-25T19:11:59.695+0800 [bookkeeper-10-3-2] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - connection [id: 0x62b69d34, L:/10.10.86.247:39422 - R:10.10.86.248/10.10.86.2
48:3181] authenticated as BookKeeperPrincipal[ANONYMOUS]
2023-06-25T19:11:59.695+0800 [bookkeeper-10-3-1] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - connection [id: 0xida31afc, L:/10.10.86.247:51494 - R:10.10.86.249/10.10.86.2
49:3181] authenticated as BookKeeperPrincipal[ANONYMOUS]
2023-06-25T19:11:59.717+0800 [main] INFO org.apache.bookkeeper.tools.cli.commands.client.SimpleTestCommand - 3 entries written to ledger 7
2023-06-25T19:11:59.737+0800 [main] ERROR org.apache.bookkeeper.client.LedgerHandle - ReadAsync exception on ledgerId:7 firstEntry:0 lastEntry:3 lastAddConfirmed:1
2023-06-25T19:11:59.779+0800 [main] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - closing the per channel bookie client for 10.10.86.247:3181
2023-06-25T19:11:59.784+0800 [main] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - closing the per channel bookie client for 10.10.86.247:3181
2023-06-25T19:11:59.785+0800 [main] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - closing the per channel bookie client for 10.10.86.248:3181
2023-06-25T19:11:59.785+0800 [bookkeeper-10-3-2] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - Disconnected from bookie channel [id: 0xcce08be7a, L:/10.10.86.247:48294 ! R:1
0.10.86.247/10.10.86.247:3181]
2023-06-25T19:11:59.785+0800 [bookkeeper-10-3-1] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - Disconnected from bookie channel [id: 0x62b69d34, L:/10.10.86.247:39422 ! R:1
0.10.86.248/10.10.86.248:3181]
2023-06-25T19:11:59.785+0800 [main] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - closing the per channel bookie client for 10.10.86.248:3181
2023-06-25T19:11:59.786+0800 [main] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - closing the per channel bookie client for 10.10.86.249:3181
2023-06-25T19:11:59.786+0800 [bookkeeper-10-3-1] INFO org.apache.bookkeeper.proto.PerChannelBookieClient - Disconnected from bookie channel [id: 0xida31afc, L:/10.10.86.247:51494 ! R:1
0.10.86.249/10.10.86.249:3181]
2023-06-25T19:11:59.786+0800 [main] WARN org.apache.bookkeeper.client.BookKeeper - The mainWorkerPool did not shutdown cleanly
2023-06-25T19:11:59.906+0800 [main] INFO org.apache.zookeeper.ZooKeeper - Session: 0x3801e9328de0004 closed
2023-06-25T19:11:59.906+0800 [main-EventThread] INFO org.apache.zookeeper.client.Cnxn - EventThread shut down for session: 0x3801e9328de0004
2023-06-25T19:11:59.907+0800 [main] ERROR org.apache.bookkeeper.tools.cli.commands.client.SimpleTestCommand - Failed to process command 'simpletest'
org.apache.bookkeeper.client.BookException$BookException: Error while reading ledger
    at org.apache.bookkeeper.client.LedgerHandle.readAsync(LedgerHandle.java:754) ~[org.apache.bookkeeper-bookkeeper-server-4.16.1.jar:4.16.1]
    at org.apache.bookkeeper.client.api.ReadHandle.read(ReadHandle.java:58) ~[org.apache.bookkeeper-bookkeeper-server-4.16.1.jar:4.16.1]
    at org.apache.bookkeeper.tools.cli.commands.client.SimpleTestCommand.run(SimpleTestCommand.java:115) ~[org.apache.bookkeeper-bookkeeper-server-4.16.1.jar:4.16.1]
    at org.apache.bookkeeper.tools.cli.commands.client.SimpleTestCommand.run(SimpleTestCommand.java:48) ~[org.apache.bookkeeper-bookkeeper-server-4.16.1.jar:4.16.1]
    at org.apache.bookkeeper.tools.cli.helpers.ClientCommand.apply(ClientCommand.java:66) ~[org.apache.bookkeeper-bookkeeper-server-4.16.1.jar:4.16.1]
    at org.apache.bookkeeper.tools.cli.helpers.ClientCommand.apply(ClientCommand.java:60) ~[org.apache.bookkeeper-bookkeeper-server-4.16.1.jar:4.16.1]
    at org.apache.bookkeeper.bookie.BookieShell$SimpleTestCmd.runCmd(BookieShell.java:914) ~[org.apache.bookkeeper-bookkeeper-server-4.16.1.jar:4.16.1]
    at org.apache.bookkeeper.bookie.BookieShell$MyCommand.runCmd(BookieShell.java:248) ~[org.apache.bookkeeper-bookkeeper-server-4.16.1.jar:4.16.1]
    at org.apache.bookkeeper.bookie.BookieShell.run(BookieShell.java:2349) ~[org.apache.bookkeeper-bookkeeper-server-4.16.1.jar:4.16.1]
    at org.apache.bookkeeper.bookie.BookieShell.main(BookieShell.java:2446) ~[org.apache.bookkeeper-bookkeeper-server-4.16.1.jar:4.16.1]
[root@ks-master bk]#
    
```

3.3.4. 部署Broker集群

3.3.4.1. 修改Broker配置文件

1. 分别进入3台服务器的/home/TLQ-CN目录。

```
cd /home/TLQ-CN
```

2. 修改broker.conf配置文件。

```
vi conf/broker.conf
```

具体配置如下：

```

.....

metadataStoreUrl=zk:ip1:2181,ip2:2181,ip3:2181

configurationMetadataStoreUrl= zk:ip1:2181,ip2:2181,ip3:2181

advertisedAddress=本机ip地址

clusterName=<cluster-Name>

licenseIp=License Server服务器ip:端口

licensePublicKey=产品public key

loadBalancerOverrideBrokerNicSpeedGbps=0.8

.....
    
```

参数说明如下表所示：

参数	说明
clusterName	集群名称，与初始化Zookeeper元数据时的集群名称相同。参见 确定集群名称 章节相关介绍。
metadataStoreUrl	元数据存储。Zookeeper地址。
configurationMetadataStoreUrl	配置元数据存储。Zookeeper地址。
advertisedAddress	本机ip地址。
licenseIps（可选）	License Server服务器IP地址和端口。POC测试可不配置。
licensePublicKey（可选）	TongLINK/Q-CN产品的公钥，可以从License Server管理控制台获取，详细介绍请参考《License Server 产品用户指南》中“授权证书管理”章节。POC测试可不配置。

配置示例如下：

```

metadataStoreUrl=zk:10.10.86.247:2181,10.10.86.248:2181,10.10.86.249:2181

configurationMetadataStoreUrl= zk:10.10.86.247:2181,10.10.86.248:2181,10.10.86.249:2181

advertisedAddress=本机ip地址

clusterName=tlq-bj-log-test

licenseIps=10.10.81.174:8888

licensePublicKey=

loadBalancerOverrideBrokerNicSpeedGbps=0.8

```

3.3.4.2. 启停Broker

• 启动Broker

◦ 分别在3个节点所在服务器的/home/TLQ-CN目录下，运行以下命令启动broker：

▪ 前台启动：

bin/tong broker

▪ 后台启动：

bin/tong-daemon start broker


```
[xxx@master1 TLQ-CN]# bin/tong-daemon start broker
```

```
doing start broker ...
```

```
starting broker, logging to /home/TLQ-CN/logs/pulsar-broker-k8s-worker1.log
```

Note: Set immediateFlush to true in conf/log4j2.yaml will guarantee the logging event is flushing to disk immediately. The default behavior is switched off due to performance considerations

启动日志中输出starting broker, logging to /home/TLQ-CN/logs/pulsar-broker-k8s-worker1.log。其中日志输出到/home/TLQ-CN/logs/pulsar-broker-k8s-worker1.log文件中，3个节点输出的日志文件命名有所不同，日志文件命名规则为：pulsar-broker-{hostname}.log，可在该文件中查看日志。

注意：如果在启动过程中出现如下错误：

```
2024-03-15T02:47:12,689+0800 [main] ERROR org.apache.pulsar.broker.loadbalance.LinuxInfoUtils - [LinuxInfo] Failed to get total nic limit.
java.io.IOException: 无效的参数
    at sun.nio.ch.FileDispatcherImpl.read0(Native Method) ~[?:?]
    at sun.nio.ch.FileDispatcherImpl.read(FileDispatcherImpl.java:48) ~[?:?]
    at sun.nio.ch.IOUtil.readIntoNativeBuffer(IOUtil.java:330) ~[?:?]
    at sun.nio.ch.IOUtil.read(IOUtil.java:296) ~[?:?]
    at sun.nio.ch.IOUtil.read(IOUtil.java:273) ~[?:?]
```

解决方法：请在conf/broker.conf中根据实际网卡速率配置如下参数：

loadBalancerOverrideBrokerNicSpeedGbps=0.8

- 查看集群节点情况：

进入/home/TLQ-CN目录，运行**bin/tong-admin brokers list \${clusterName}**，将\${clusterName}替换为所部署集群的名称，如果列表包含所有节点，则代表Broker部署成功。

```
[xxx@master1 TLQ-CN]# bin/tong-admin brokers list tlq-cn
```

```
Warning:Nashorn engine is planned to be removed from a future JDK release
```

```
10.10.86.247:8080
```

```
10.10.86.248:8080
```

```
10.10.86.249:8080
```

- 也可以通过查看是否有Broker的端口6650和8080确认Broker是否部署成功：

netstat -lntp |grep -E '6650 | 8080'

• （可选）停止Broker服务

- 以前台方式启动，分别在3个节点启动程序运行窗口直接按下Ctrl+C即可停止Broker。
- 以后台方式启动，分别在3个节点所在服务器的/home/TLQ-CN目录下，运行以下命令停止Broker：

bin/tong-daemon stop broker

```
[xxx@master1 TLQ-CN]# bin/tong-daemon stop broker

doing stop broker ...

stopping broker

Shutdown is in progress... Please wait...

Shutdown completed.
```

3.3.5. 集群可用性测试

1. 进入任意一台服务器的安装目录，如进入zk的安装目录/home/TLQ-CN。

cd /home/TLQ-CN

2. 修改conf文件夹下**client.conf**配置文件。

vi conf/client.conf

```
webServiceUrl=http://host1:8080,host2:8080,host3:8080

brokerServiceurl=pulsar://host1:6650,host2:6650,host3:6650
```

配置示例如下：

```
webServiceUrl=http://10.10.86.247:8080,10.10.86.248:8080,10.10.86.249:8080

brokerServiceUrl=pulsar://10.10.86.247:6650,10.10.86.248:6650,10.10.86.249:6650
```

3. 在/home/TLQ-CN目录下输入以下命令发布消息：

```
bin/tong-client produce \

persistent://public/default/test \

-n 1 \

-m "Hello Pulsar"
```

结果如下：

```
[root@8s-master zk]# bin/tong-client produce \
> persistent://public/default/test \
> -n 1 \
> -m "Hello Pulsar"

2023-06-25T19:53:27,919+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ConnectionPool - [[id: 0x0be33c50, L:/10.10.86.247:32950 - R:10.10.86.248/10.10.86.248:6650]] Connected to server
2023-06-25T19:53:28,393+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - Starting Pulsar producer perf with config: {"topicName":"persistent://public/default/test","producerName":null,"sendTimeoutMs":30000,"blockIfQueueFull":false,"maxPendingMessages":1000,"maxPendingMessagesAcrossPartitions":50000,"messageRoutingMode":"RoundRobinPartition","hashingScheme":"JavaStringHash","cryptoFailureAction":"FAIL","batchingMaxPublishDelayMicros":1000,"batchingPartitionsSwitchFrequencyPublishDelay":10,"batchingMaxMessages":1000,"batchingMaxBytes":131072,"batchingEnabled":true,"chunkingEnabled":false,"chunkMaxMessageSize":1,"encryptionKeys":[],"compressionType":"NONE","initialSequenceId":null,"autoUpdatePartitions":true,"autoUpdatePartitionsIntervalSeconds":60,"multiSchema":true,"accessMode":"Shared","lazyStartPartitionedProducers":false,"properties":{},"initialSubscriptionName":null}
2023-06-25T19:53:28,409+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - Pulsar client config: {"serviceUrl":"pulsar://10.10.86.247:6650,10.10.86.248:6650,10.10.86.249:6650","authPluginClassName":null,"authParams":null,"authParamMap":null,"operationTimeoutMs":30000,"lookupTimeoutMs":30000,"statsIntervalSeconds":60,"numIoThreads":8,"numListenerThreads":8,"connectionsPerBroker":1,"connectionMaxIdleSeconds":180,"useTcpNoDelay":true,"useTls":false,"tlsKeyFilePath":"","tlsCertificateFilePath":"","tlsTrustCertificatePath":"","tlsAllowInsecureConnection":false,"tlsHostnameVerificationEnabled":false,"concurrentLookupRequest":5000,"maxLookupRequest":50000,"maxLookupRedirects":20,"maxNumberOfRejectedRequestPerConnection":50,"keepAliveIntervalSeconds":30,"connectionTimeoutMs":10000,"requestTimeoutMs":60000,"readTimeoutMs":60000,"autoCertRefreshSeconds":300,"initialBackoffIntervalNanos":1000000000,"maxBackoffIntervalNanos":60000000000,"enableBusyWait":false,"listenerName":null,"useKeyStoreTls":false,"sslProvider":null,"tlsKeyStoreType":"JKS","tlsKeyStorePath":"","tlsKeyStorePassword":"*****","tlsTrustStoreType":"JKS","tlsTrustStorePath":"","tlsTrustStorePassword":"*****","tlsCipherSuites":[],"tlsProtocols":[],"memoryLimitBytes":0,"proxyServiceUrl":null,"proxyProtocol":null,"enableTransaction":false,"dnsLookupBindAddress":null,"dnsLookupBindPort":0,"socks5ProxyAddress":null,"socks5ProxyUsername":null,"socks5ProxyPassword":null,"description":null}
2023-06-25T19:53:28,413+0800 [pulsar-client-io-1-5] INFO org.apache.pulsar.client.impl.ConnectionPool - [[id: 0xe43d7c12, L:/10.10.86.247:54636 - R:10.10.86.249/10.10.86.249:6650]] Connected to server
2023-06-25T19:53:28,604+0800 [pulsar-client-io-1-7] INFO org.apache.pulsar.client.impl.ConnectionPool - [[id: 0xb8deaf34, L:/10.10.86.247:59620 - R:10.10.86.247/10.10.86.247:6650]] Connected to server
2023-06-25T19:53:28,924+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ProducerImpl - [persistent://public/default/test] [null] Creating producer on cnx [id: 0x0be33c50, L:/10.10.86.247:32950 - R:10.10.86.248/10.10.86.248:6650]
2023-06-25T19:53:29,212+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ProducerImpl - [persistent://public/default/test] [tlq-cn-1-0] Created producer on cnx [id: 0x0be33c50, L:/10.10.86.247:32950 - R:10.10.86.248/10.10.86.248:6650]
2023-06-25T19:53:29,455+0800 [main] INFO org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - [persistent://public/default/test] [tlq-cn-1-0] --- Publish throughput: 0.96 msg/s --- 0.00 Mbit/s --- Latency: med: 227.000 ms - 95pct: 227.000 ms - 99pct: 227.000 ms - max: 227.000 ms --- BatchSize: med: 1.000 - 95pct: 1.000 - 99pct: 1.000 - max: 12.000 bytes --- MsgSize: med: 12.000 bytes - 95pct: 12.000 bytes - 99pct: 12.000 bytes --- Ack received rate: 0.96 ack/s --- Fail messages: 0 --- Pending messages: 0
2023-06-25T19:53:29,462+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ProducerImpl - [persistent://public/default/test] [tlq-cn-1-0] Closed Producer
2023-06-25T19:53:29,464+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ProducerImpl - Client closing, URL: pulsar://10.10.86.247:6650,10.10.86.248:6650,10.10.86.249:6650
2023-06-25T19:53:29,472+0800 [pulsar-client-io-1-5] INFO org.apache.pulsar.client.impl.ClientCnx - [id: 0xe43d7c12, L:/10.10.86.247:54636 ! R:10.10.86.249/10.10.86.249:6650] Disconnect
2023-06-25T19:53:29,472+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ClientCnx - [id: 0x0be33c50, L:/10.10.86.247:32950 ! R:10.10.86.248/10.10.86.248:6650] Disconnect
2023-06-25T19:53:29,472+0800 [pulsar-client-io-1-7] INFO org.apache.pulsar.client.impl.ClientCnx - [id: 0xb8deaf34, L:/10.10.86.247:59620 ! R:10.10.86.247/10.10.86.247:6650] Disconnect
2023-06-25T19:53:31,486+0800 [main] INFO org.apache.pulsar.client.cli.PulsarClientTool - 1 messages successfully produced
[root@8s-master zk]#
```

4. 打开另一个终端会话，在/home/TLQ-CN目录下输入以下命令消费消息：

```
bin/tong-client consume \
persistent://public/default/test \
-n 100 \
-s "consumer-test" \
-t "Exclusive"
```

结果如下：（第一次消费时，由于没有创建订阅，所以无法消费到消息。使用生产者再次发送消息后即可消费到消息。）

```
2023-06-25T19:55:14,822+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ConnectionPool - [[id: 0x378a5e09, L:/10.10.86.247:49414 - R:10.10.86.247/10.10.86.247:6650]] Connected to server
2023-06-25T19:55:15,069+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ConsumerStatsRecorderImpl - Starting Pulsar consumer status recorder with config: {"topicNames":["persistent://public/default/test"],"topicsPattern":null,"subscriptionName":"consumer-test","subscriptionType":"Exclusive","subscriptionProperties":null,"subscriptionMode":"Durable","receiverQueueSize":1000,"acknowledgementsGroupTimeoutMicros":1000000,"maxAcknowledgeGroupSize":1000,"negativeAckRedeliveryDelayMicros":600000000,"maxTotalReceiverQueueSizeAcrossPartitions":50000,"consumerName":null,"ackTimeoutMillis":0,"tickDurationMillis":1000,"priorityLevel":0,"maxPendingChunkedMessage":10,"autoAcknowledgeChunkedMessageOnQueueFull":false,"expireTimeOfIncompleteChunkedMessageMillis":60000,"cryptoFailureAction":"FAIL","properties":{},"readCompacted":false,"subscriptionInitialPosition":"Latest","patternAutoDiscoveryPeriod":60,"regexSubscriptionMode":"PersistentOnly","deadLetterPolicy":null,"retryEnable":false,"autoUpdatePartitions":true,"autoUpdatePartitionsIntervalSeconds":60,"replicateSubscriptionState":false,"resetInfiniteHead":false,"batchIndexAckEnabled":false,"ackReceiptEnabled":false,"poolMessages":true,"startPaused":false,"autoScaledReceiverQueueSizeEnabled":false,"topicConfigurations":{},"maxPendingChunkedMessage":10}
2023-06-25T19:55:15,086+0800 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ConsumerStatsRecorderImpl - Pulsar client config: {"serviceUrl":"pulsar://10.10.86.247:6650,10.10.86.248:6650,10.10.86.249:6650","authPluginClassName":null,"authParams":null,"authParamMap":null,"operationTimeoutMs":30000,"lookupTimeoutMs":30000,"statsIntervalSeconds":60,"numIoThreads":8,"numListenerThreads":8,"connectionsPerBroker":1,"connectionMaxIdleSeconds":180,"useTcpNoDelay":true,"useTls":false,"tlsKeyFilePath":"","tlsCertificateFilePath":"","tlsTrustCertificatePath":"","tlsAllowInsecureConnection":false,"tlsHostnameVerificationEnabled":false,"concurrentLookupRequest":5000,"maxLookupRequest":50000,"maxLookupRedirects":20,"maxNumberOfRejectedRequestPerConnection":50,"keepAliveIntervalSeconds":30,"connectionTimeoutMs":10000,"requestTimeoutMs":60000,"readTimeoutMs":60000,"autoCertRefreshSeconds":300,"initialBackoffIntervalNanos":1000000000,"maxBackoffIntervalNanos":60000000000,"enableBusyWait":false,"listenerName":null,"useKeyStoreTls":false,"sslProvider":null,"tlsKeyStoreType":"JKS","tlsKeyStorePath":"","tlsKeyStorePassword":"*****","tlsTrustStoreType":"JKS","tlsTrustStorePath":"","tlsTrustStorePassword":"*****","tlsCipherSuites":[],"tlsProtocols":[],"memoryLimitBytes":0,"proxyServiceUrl":null,"proxyProtocol":null,"enableTransaction":false,"dnsLookupBindAddress":null,"dnsLookupBindPort":0,"socks5ProxyAddress":null,"socks5ProxyUsername":null,"socks5ProxyPassword":null,"description":null}
2023-06-25T19:55:15,094+0800 [pulsar-client-io-1-6] INFO org.apache.pulsar.client.impl.ConnectionPool - [[id: 0xc9e383f4, L:/10.10.86.247:35830 - R:10.10.86.248/10.10.86.248:6650]] Connected to server
2023-06-25T19:55:15,111+0800 [pulsar-client-io-1-6] INFO org.apache.pulsar.client.impl.ConsumerImpl - [persistent://public/default/test][consumer-test] Subscribing to topic on cnx [id: 0xc9e383f4, L:/10.10.86.247:35830 - R:10.10.86.248/10.10.86.248:6650], consumerId 0
2023-06-25T19:55:15,121+0800 [pulsar-client-io-1-6] INFO org.apache.pulsar.client.impl.ConsumerImpl - [persistent://public/default/test][consumer-test] Subscribed to topic on 10.10.86.248/10.10.86.248:6650, consumerId 0
---- got message ----
key:[null], properties:{}, content:Hello Pulsar
```

3.3.6. 服务地址

部署TongLINK/Q-CN集群后，对外提供的服务地址（需将IP地址修改为实际所使用的3个节点的IP地址）如下表所示：

- 业务地址：客户端收发消息时所使用的服务端地址；
- 管理地址：管理控制台管理该集群时配置的地址。

作用	地址
业务地址	broker-service-url pulsar://10.10.86.247:6650,10.10.86.248:6650,10.10.86.249:6650
管理地址	web-service-url http://10.10.86.247:8080,10.10.86.248:8080,10.10.86.249:8080

附录 A: 术语说明

名称	说明
TongLINK/Q-CN (简称TLQ-CN)	东方通云原生消息中间件软件
Broker	服务端工作节点
Client	客户端
Topic	主题
Message	消息
Producer	生产者，即消息的提供者
Consumer	消费者，即消息的消费者
Pub-Sub	发布及订阅
Point-to-Point(PTP)	点对点模式，即生产者和消费者之间的消息往来

